NumPy Basics: Arrays and Vectorized Computation

1. Introduction to NumPy

What is NumPy?

NumPy stands for **Numerical Python**. It is the foundational core library in Python used for efficient numerical and scientific computing.

NumPy is primarily applied in:

- Numerical calculations
- High-performance array operations
- Mathematical transformations
- Data Analysis & Scientific Computing
- Machine Learning & Deep Learning algorithms

When handling massive amounts of numerical data, standard Python lists become computationally inefficient. NumPy introduces a specialized, contiguous memory data structure called an **array** specifically designed for numerical processing.

Importing NumPy

To use NumPy in Python, it must first be imported. By industry convention, numpy is aliased as `np`:

```
import numpy as np
```

- `import` tells Python to load the library.
- `numpy` is the name of the package.
- `as np` assigns a shortened standard alias so you can write `np.array()` instead of `numpy.array()`.

2. Python Lists vs. NumPy Arrays

Creating an Array

```
import numpy as np

# Creating a NumPy array from a list

marks = np.array([70, 80, 65, 90, 75])

print(marks)

# Output: [70 80 65 90 75]
```

Key Differences

Difference 1: Scalar Multiplication / Numerical Operations

For standard Python lists, the * operator performs sequence repetition rather than mathematical multiplication.

```
# Python List Multiplication
list_marks = [10, 20, 30, 40]
print(list_marks * 2)
# Output: [10, 20, 30, 40, 10, 20, 30, 40]  (Repetition)

# NumPy Array Multiplication
arr_marks = np.array([10, 20, 30, 40])
print(arr_marks * 2)
# Output: [20 40 60 80]  (Element-wise Multiplication)
```

Difference 2: Element-wise Arithmetic

Adding two standard Python lists concatenates them into a single longer list.
Adding two NumPy arrays performs mathematical element-wise addition.

```
# Python List Addition
a_list = [10, 20, 30]
b_list = [1, 2, 3]
print(a_list + b_list)
# Output: [10, 20, 30, 1, 2, 3]  (Concatenation)

# NumPy Array Addition
a_arr = np.array([10, 20, 30])
b_arr = np.array([1, 2, 3])
print(a_arr + b_arr)
# Output: [11 22 33]  (Element-wise Addition)
```

3. Array Dimensions & Visual Representations

NumPy arrays can exist in multiple dimensions:

1D Array (Vector)

A one-dimensional collection of elements arranged in a single line or row.

```
arr_1d = np.array([10, 20, 30, 40])
```

Visual Model:



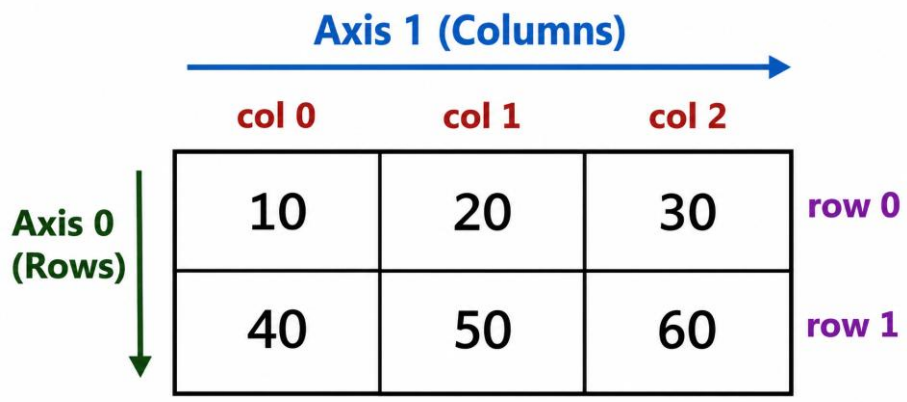
2D Array (Matrix)

A two-dimensional grid composed of rows and columns, similar to a spreadsheet table or mathematical matrix.

```
arr_2d = np.array([
    [10, 20, 30],
    [40, 50, 60]
```

```
])
```

Visual Model:

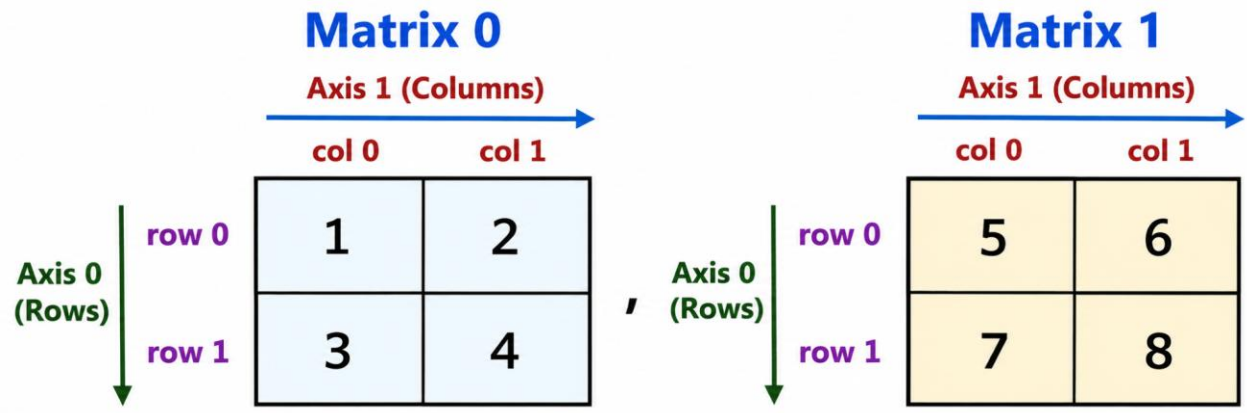


3D Array (Tensor / Stacked Matrices)

A three-dimensional structure that can be visualized as multiple 2D tables stacked along a depth axis.

```
arr_3d = np.array([
    [[1, 2], [3, 4]],
    [[5, 6], [7, 8]]
])
```

Visual Model:



4. Key Terminology & Essential Array Properties

Core Terminology

- **Dimension**: An axis along which data elements are stored and accessed.
- **Axis**: Directional reference for indexing and operations.
    - `axis = 0`: Operates downwards across rows.
    - `axis = 1`: Operates across columns.

Primary Array Attributes

```
arr = np.array([
    [10, 20, 30],
    [40, 50, 60]
])
```

Property	Description	Example Output for arr
`ndim`	Returns the number of dimensions/axes.	`2`
`shape`	Returns a tuple indicating array dimensions `(rows, columns)`.	`(2, 3)`
`size`	Returns the total count of elements inside the array.	`6`
`dtype`	Identifies the data type of the stored elements.	`int64` (or `int32`)

Property Quick Reference

- **ndim**: How many dimensions?
- **shape**: What is the structural layout?

- **size**: How many total items?
- **dtype**: What type of data is stored?

5. Vectorized Computation

What is Vectorization?

Vectorization is the capability of executing high-level mathematical operations across an entire array simultaneously without using explicit Python `for` loops.

Loop Approach vs. Vectorized Approach

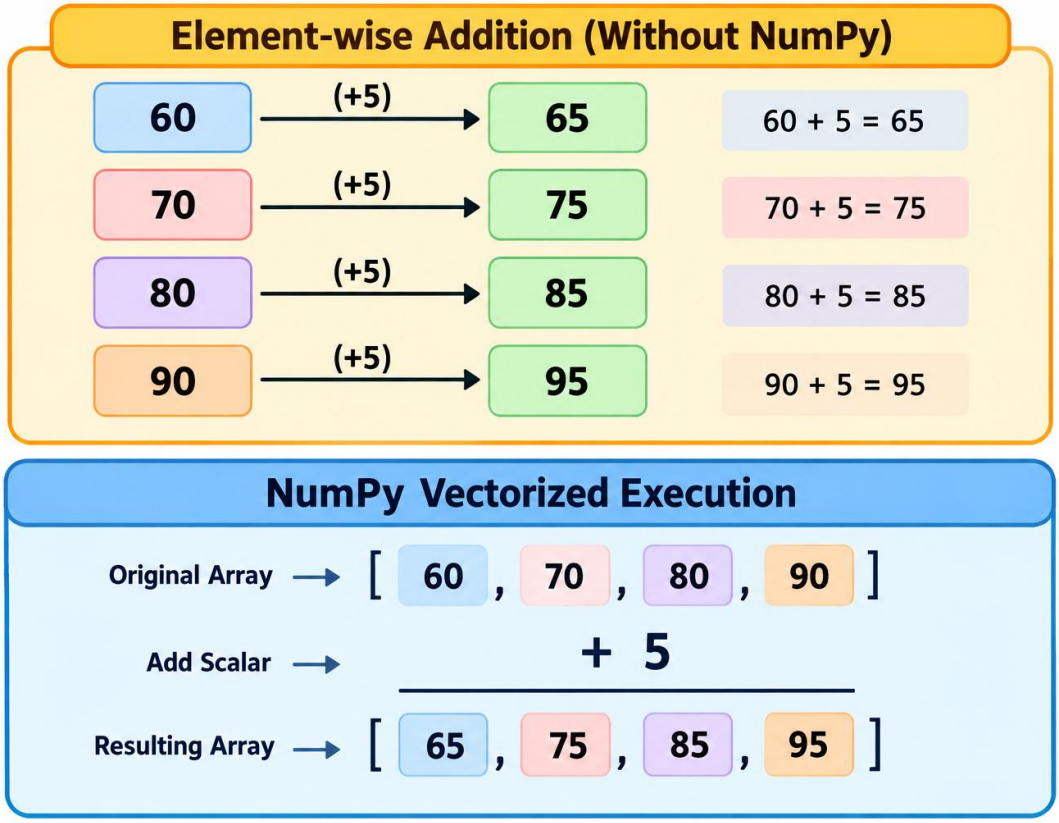
Loop Approach (Standard Python):

```
marks = [60, 70, 80, 90]
new_marks = []
for mark in marks:
    new_marks.append(mark + 5)
```

Vectorized Approach (NumPy):

```
marks = np.array([60, 70, 80, 90])
new_marks = marks + 5  # Applied instantly to all elements
```

Visual Execution Diagram



Arithmetic Operations

-

```
a = np.array([10, 20, 30])
b = np.array([1, 2, 3])
```

- **Addition (+)**: `a + b` $\rightarrow$ `[11, 22, 33]`
- **Subtraction (-)**: `a - b` $\rightarrow$ `[9, 18, 27]`
- **Multiplication (*)**: `a * b` $\rightarrow$ `[10, 40, 90]`
- **Division (/)**: `a / b` $\rightarrow$ `[10.0, 10.0, 10.0]`

Scalar Operations

A **scalar** is a single constant value (e.g., 5). Applying a scalar to a NumPy array broadcasts the value to every single element:

```
arr = np.array([10, 20, 30, 40])
print(arr + 5)   # Output: [15 25 35 45]
```

```
print(arr * 2)   # Output: [20 40 60 80]
print(arr - 5)   # Output: [5 15 25 35]
print(arr / 10)  # Output: [1. 2. 3. 4.]
```

Benefits of Vectorization

1. **Cleaner Code**: Eliminates verbose loop constructs.
2. **Readability**: Code closely resembles mathematical notations.
3. **High Performance**: Executed using optimized internal C-level compiled subroutines.
4. **Data Science Standard**: Basis for higher-level libraries including Pandas, SciPy, and TensorFlow/PyTorch.

6. Practical Real-World Examples

Case Study 1: Academic Grading Adjustments

```
import numpy as np
marks = np.array([65, 72, 80, 55, 90])

# Scenario A: Add 5 grace marks
grace_marks = marks + 5
print("Grace Marks:", grace_marks)
# Output: [70 77 85 60 95]

# Scenario B: 10% percentage boost
boosted_marks = marks * 1.10
print("Boosted Marks:", boosted_marks)
# Output: [71.5 79.2 88.  60.5 99. ]

# Scenario C: 10% penalty reduction
reduced_marks = marks * 0.90
print("Reduced Marks:", reduced_marks)
```

```
# Output: [58.5 64.8 72.  49.5 81. ]
```

Case Study 2: Commercial Pricing & Tax (GST) Calculations

```
import numpy as np
prices = np.array([100, 200, 300, 400])

# Compute 18% GST tax amount for each product
gst_amount = prices * 0.18
print("GST Component:", gst_amount)
# Output: [18. 36. 54. 72.]

# Calculate final customer prices (Base Price + 18%)
final_prices = prices * 1.18
print("Final Prices:", final_prices)
# Output: [118. 236. 354. 472.]
```

7. Hands-On Practical Exercises

Exercise 1: Array Attribute Inspection

Create an array `arr = np.array([10, 20, 30, 40, 50])` and display its core attributes.

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])
print("Array:", arr)          # [10 20 30 40 50]
print("NDim:", arr.ndim)      # 1
print("Shape:", arr.shape)    # (5,)
print("Size:", arr.size)      # 5
print("DType:", arr.dtype)    # int64
```

Exercise 2: Scalar Arithmetic

Given `a = np.array([10, 20, 30, 40])`, calculate `+5`, `*2`, `-3`, and `/2`.

```
a = np.array([10, 20, 30, 40])
print(a + 5)  # [15 25 35 45]
print(a * 2)  # [20 40 60 80]
print(a - 3)  # [ 7 17 27 37]
print(a / 2)  # [ 5. 10. 15. 20.]
```

Exercise 3: Multi-Array Operations

Given `a = np.array([10, 20, 30])` and `b = np.array([2, 4, 5])`, predict and compute basic arithmetic operations.

```
a = np.array([10, 20, 30])
b = np.array([2, 4, 5])
print(a + b)  # [12 24 35]
print(a - b)  # [ 8 16 25]
print(a * b)  # [ 20  80 150]
print(a / b)  # [5. 5. 6.]
```

Exercise 4: 2D Matrix Vectorization

```
arr_2d = np.array([
    [10, 20, 30],
    [40, 50, 60]
])

print("ndim:", arr_2d.ndim)    # Output: 2
print("shape:", arr_2d.shape)  # Output: (2, 3)
print("size:", arr_2d.size)    # Output: 6
```

```
# 2D Vectorized Addition
print(arr_2d + 10)
# Output:
# [[20 30 40]
#  [50 60 70]]
```

8. Frequently Asked Questions (FAQ)

Q1: Is NumPy a standalone programming language?

No, NumPy is an open-source extension library for the Python programming language.

Q2: How does a NumPy array differ from a standard Python list?

While standard lists hold generic object pointers and support heterogeneous items, NumPy arrays store homogenous data sequentially in memory, enabling vectorized calculations.

Q3: Can NumPy arrays store non-numeric data like strings?

Yes, NumPy arrays can hold text strings, but the library is optimized specifically for numerical and scientific routines.

```
str_arr = np.array(["Python", "Java", "C++"])
print(str_arr.dtype)  # Output: <U6 (Unicode string)
```

Q4: What happens when mixed data types are passed into `np.array()`?

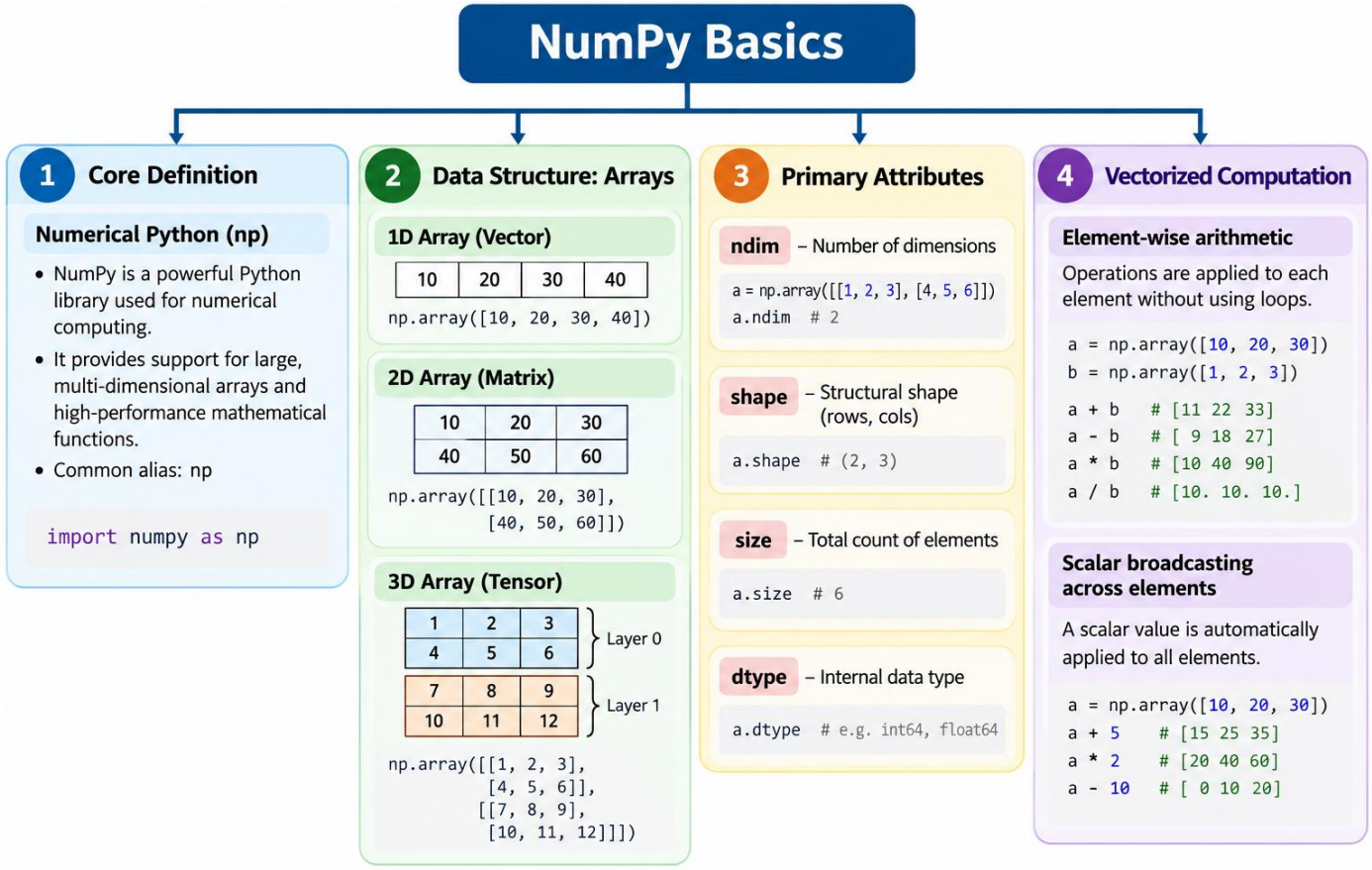
NumPy automatically upcasts the elements to a single common compatible `dtype`. For instance, mixing integers and floating-point numbers converts all entries to floats:

```
mixed_arr = np.array([1, 2, 3.5])
print(mixed_arr.dtype)  # Output: float64
print(mixed_arr)        # Output: [1.  2.  3.5]
```

Q5: Why does dividing integers with `/` result in decimals?

In Python and NumPy, standard division `/` always returns floating-point values to maintain mathematical accuracy.

9. Conceptual Mind Map Summary



10. Indexing, Slicing & Subsetting

Indexing and slicing allow accessing and modifying specific elements or sub-regions of NumPy arrays efficiently.

1D Array Indexing & Slicing

1D arrays support standard Python zero-based indexing and slice notation `[start:stop:step]`.

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50])

# Indexing
print(arr[0])      # Output: 10
print(arr[-1])     # Output: 50
```

```
# Slicing
print(arr[1:4])    # Output: [20 30 40]
print(arr[::2])    # Output: [10 30 50]
```

2D Array Indexing & Slicing

2D arrays use comma-separated tuple indexing `[row_index, column_index]` or range slices `[row_start:row_stop, col_start:col_stop]`.

```
matrix = np.array([
    [10, 20, 30],
    [40, 50, 60],
    [70, 80, 90]
])

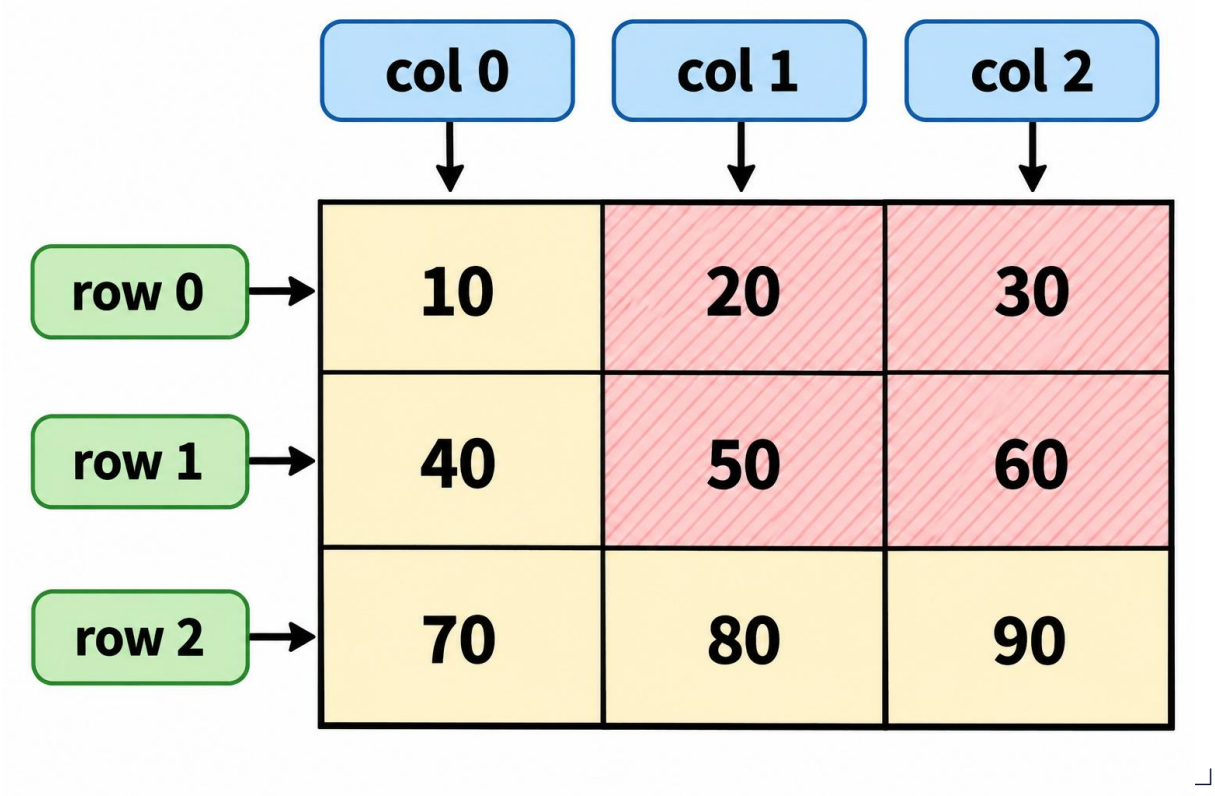
# Specific element: Row index 1, Column index 2
print(matrix[1, 2])     # Output: 60

# Extracting an entire row or column
print(matrix[0, :])     # Row 0: [10 20 30]
print(matrix[:, 1])     # Column 1: [20 50 80]

# Sub-matrix slicing (Rows 0 to 1, Columns 1 to 2)
print(matrix[0:2, 1:3])
# Output:
# [[20 30]
#  [50 60]]
```

Visual Slicing Diagram

Slicing `matrix[0:2, 1:3]`:



Boolean Indexing (Filtering)

Boolean indexing filters array elements using comparison masks.

```
arr = np.array([12, 45, 67, 23, 89, 34])

# Create boolean mask for values > 40
mask = arr > 40
print(mask)        # [False  True  True False  True False]

# Apply boolean index mask
filtered = arr[mask]
print(filtered)    # Output: [45 67 89]
```

11. Array Reshaping & Transposition

Reshaping changes array dimensions without altering the underlying data.

Reshaping with reshape()

```
arr_1d = np.array([1, 2, 3, 4, 5, 6])

# Reshape 1D array into 2x3 2D matrix
arr_2d = arr_1d.reshape(2, 3)
print(arr_2d)
# Output:
# [[1 2 3]
#  [4 5 6]]

# Automatic dimension inference using -1
arr_inferred = arr_1d.reshape(3, -1)  # Infers 2 columns
print(arr_inferred.shape)            # Output: (3, 2)
```

Transposing with .T or transpose()

Transposition flips an array over its diagonal, swapping row and column indices.

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])

transposed = matrix.T
print(transposed)
# Output:
# [[1 4]
#  [2 5]
#  [3 6]]
```

12. Broad Broadcasting Mechanism

Broadcasting describes how NumPy treats arrays of different shapes during arithmetic operations.

Rules of Broadcasting

5. **Rule 1**: If arrays differ in number of dimensions, the shape of the one with fewer dimensions is padded with ones on its left side.
6. **Rule 2**: If the shape along an axis does not match, the array with size 1 along that axis is stretched to match the larger size.
7. **Rule 3**: If along any dimension sizes disagree and neither equals 1, an error is raised.

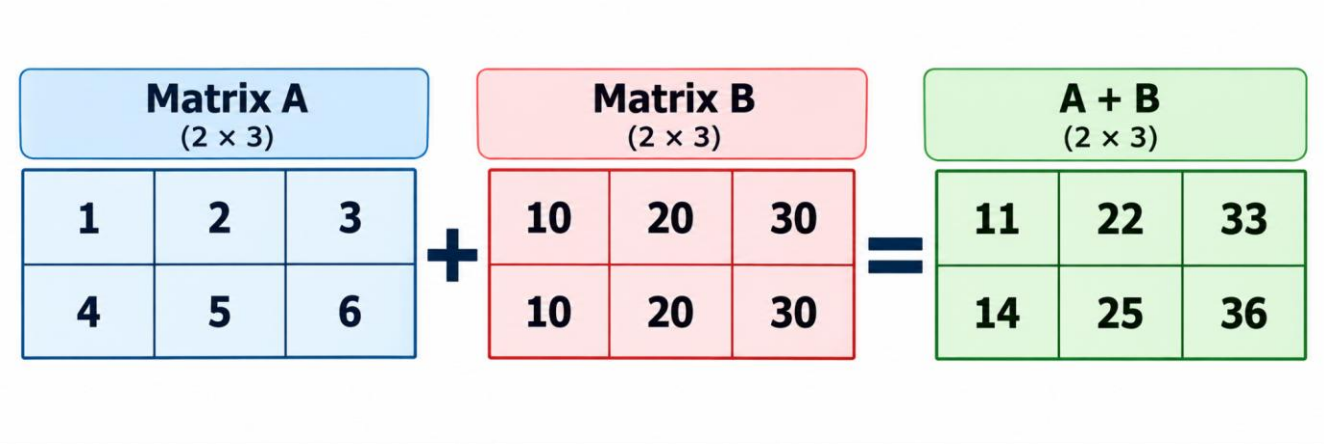
Broadcasting Example

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])  # Shape: (2, 3)

row_vector = np.array([10, 20, 30])  # Shape: (3,)

result = matrix + row_vector
print(result)
# Output:
# [[11 22 33]
#  [14 25 36]]
```

Visual Broadcasting Diagram



13. Universal Functions (ufuncs) & Aggregations

NumPy provides built-in fast element-wise functions (ufuncs) and statistical summary methods.

Common Statistical Aggregations

```
arr = np.array([10, 20, 30, 40, 50])

print("Sum:", np.sum(arr))          # Output: 150
print("Mean:", np.mean(arr))        # Output: 30.0
print("Std Dev:", np.std(arr))      # Output: 14.1421
print("Min:", np.min(arr))          # Output: 10
print("Max:", np.max(arr))          # Output: 50
print("Argmax:", np.argmax(arr))    # Output: 4 (Index of max
value)
```

Axis-wise Aggregations

```
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6]
])
```

```
# Sum along Axis 0 (column-wise reduction across rows)
print(np.sum(matrix, axis=0))  # Output: [5 7 9]

# Sum along Axis 1 (row-wise reduction across columns)
print(np.sum(matrix, axis=1))  # Output: [ 6 15]
```

14. Deep Dive into NumPy ndarray Internal Mechanics

The `ndarray` (N-dimensional array) object is the core data structure of NumPy. It consists of a raw data buffer combined with metadata describing how to interpret and access the elements.

Memory Layout & Strides

Unlike Python lists, which store references/pointers to objects scattered across system memory, a NumPy array stores data in a continuous block of memory (contiguous memory block). The array metadata object manages mapping indices to exact byte offsets.

- **Data Buffer**: A contiguous block of raw memory bytes.
- **Strides**: A tuple specifying the number of bytes to step in each dimension when moving to the next element.

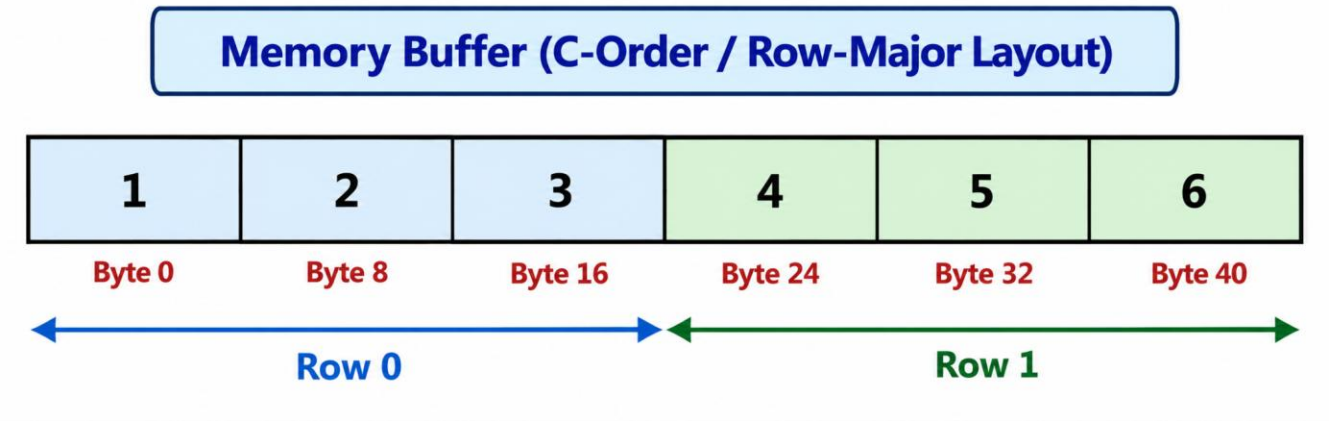
Understanding Strides

For a 2D matrix of shape `(2, 3)` with 64-bit integers (8 bytes each):

```
arr = np.array([[1, 2, 3], [4, 5, 6]], dtype=np.int64)
print(arr.strides)  # Output: (24, 8)
```

- `24 bytes`: Step needed to jump down 1 row (3 columns × 8 bytes).
- `8 bytes`: Step needed to jump across 1 column (1 item × 8 bytes).

Visualizing Contiguous Memory Layout



15. Array Creation Routines

NumPy offers built-in constructor functions to initialize arrays with different shapes and values efficiently.

Common Constructors

- `np.array()`: Converts Python sequences (lists, tuples) to an ndarray.
- `np.zeros(shape)`: Initializes an array filled completely with zeros.
- `np.ones(shape)`: Initializes an array filled completely with ones.
- `np.empty(shape)`: Allocates raw memory without initializing values (fastest, uninitialized junk values).
- `np.arange(start, stop, step)`: Generates evenly spaced values within a given interval using step size.
- `np.linspace(start, stop, num)`: Generates `num` evenly spaced samples over a specified interval.

Code Examples

```
import numpy as np

# Zeros and Ones
z = np.zeros((2, 3))     # 2x3 matrix filled with 0.0
```

```
o = np.ones((2, 2))      # 2x2 matrix filled with 1.0

# Uninitialized memory
e = np.empty((2, 2))     # Contains whatever values are in
allocated memory

# Arange vs Linspace
arr_range = np.arange(0, 10, 2)       # [0, 2, 4, 6, 8]
arr_space = np.linspace(0, 1, 5)      # [0.  , 0.25, 0.5 , 0.75,
1.  ]
```

16. Creating ndarrays: Comprehensive Guide

NumPy provides a rich set of creation routines to initialize arrays for various numerical workflows, ranging from explicit data conversion to structured sequences and random distributions.

1. Array Creation Routines Overview

- `np.array()`: Converts existing sequences (lists, tuples) into an ndarray.
- `np.zeros()` / `np.ones()`: Fills an array of specified shape with 0s or 1s.
- `np.full()`: Fills an array with a specified constant value.
- `np.eye()` / `np.identity()`: Generates a 2D identity matrix with ones on the main diagonal.
- `np.arange()`: Creates evenly spaced values based on a step size interval.
- `np.linspace()`: Creates a specific number of evenly spaced samples over a specified range.

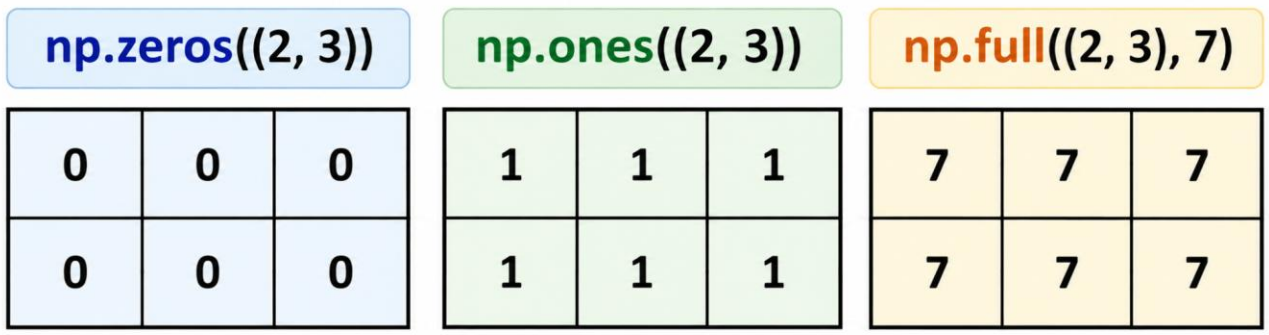
```
import numpy as np

# Basic sequence conversion
arr_from_list = np.array([1, 2, 3, 4])
```

```
# Fixed value initialization
zeros_arr = np.zeros((2, 3))
ones_arr = np.ones((2, 3))
full_arr = np.full((2, 3), 7)
identity_matrix = np.eye(3)

# Numerical range generation
range_arr = np.arange(0, 10, 2)       # [0, 2, 4, 6, 8]
linear_arr = np.linspace(0, 1, 5)      # [0.0, 0.25, 0.5, 0.75,
1.0]
```

Visual Representation of Basic Initialization Routines



2. Data Types (dtypes) & Type Casting

Every ndarray has an associated `dtype` object specifying the layout and interpretation of memory bytes.

- **Common Data Types**: `int32`, `int64`, `float32`, `float64`, `bool_`, and `string_`.
- **Type Casting with astype()**: Converts an array explicitly to a new data type, creating a fresh copy in memory.

```
# Specifying dtype at creation
float_arr = np.array([1, 2, 3], dtype=np.float64)
print("Dtype:", float_arr.dtype)  # Output: float64

# Casting array types using astype()
int_arr = float_arr.astype(np.int32)
print("Casted Dtype:", int_arr.dtype)  # Output: int32
```

3. Random Data Generation (np.random)

NumPy provides robust routines for pseudorandom number generation using the standard `np.random` module and the modern generator API (`default_rng`).

- `rand(d0, d1, ...)`: Uniform distribution over [0, 1).
- `randn(d0, d1, ...)`: Standard normal distribution (mean=0, variance=1).
- `randint(low, high, size)`: Random integers within a specified range.

```
# Legacy random API
uniform_data = np.random.rand(2, 3)
normal_data = np.random.randn(2, 3)
int_data = np.random.randint(10, 50, size=(2, 3))

# Modern Generator API (Recommended)
rng = np.random.default_rng(seed=42)
random_floats = rng.random((2, 3))
random_integers = rng.integers(10, 50, size=(2, 3))
```

17. Deep Dive: Data Types for ndarrays (dtypes)

Every NumPy array possesses a data type descriptor (`dtype`) that dictates how the underlying binary data buffer in memory is constructed, sized, and interpreted by high-level algorithms.

1. Overview of Core Data Type Categories

NumPy provides specialized primitive types optimized for scientific computing performance, extending standard Python types with explicit bit-depth precision choices:

- **Integers (Signed & Unsigned)**: Used for whole numbers. Signed integers support negative and positive values, while unsigned integers (`uint`) represent non-negative numbers only.
- **Floating-Point Numbers**: Used for real-valued decimal calculations with varying decimal precision and exponential ranges.
- **Booleans**: Binary true/false values stored as single bytes for efficient logical masking operations.
- **Complex Numbers**: Floating-point numbers composed of paired real and imaginary components.
- **String & Unicode Types**: Fixed-width byte strings (`S`) and Unicode text strings (`U`).

2. Precision, Bit Depth, and Memory Allocation Table

Data Type	Type Code	Bytes	Numerical Range / Precision
`int8`	`i1`	`1 byte`	-128 to 127
`int16`	`i2`	`2 bytes`	-32,768 to 32,767
`int32`	`i4`	`4 bytes`	-2,147,483,648 to 2,147,483,647
`int64`	`i8`	`8 bytes`	-9.22×10^{18} to 9.22×10^{18}
`uint8`	`u1`	`1 byte`	0 to 255 (Standard Image Processing)
`float16`	`f2`	`2 bytes`	Half precision (~3-4 decimal digits)

Data Type	Type Code	Bytes	Numerical Range / Precision
`float32`	`f4`	`4 bytes`	Single precision (~6-9 decimal digits)
`float64`	`f8`	`8 bytes`	Double precision (~15-17 decimal digits)
`complex128`	`c16`	`16 bytes`	Two float64 values (real and imag)
`bool_`	`b1`	`1 byte`	True or False
`<U10`	`U10`	`40 bytes`	10-character UTF-32 string (4 bytes/char)

3. Explicit Type Casting with astype()

Converting an array's data type using `astype()` creates a new array with reallocated memory.

```
import numpy as np
unittest_arr = np.array([1.2, 2.8, 3.5, 4.1])
print("Original:", unittest_arr, "| Dtype:", unittest_arr.dtype)
# Casting floats to integers truncates decimal values toward
zero
int_arr = unittest_arr.astype(np.int32)
print("Casted:", int_arr, "| Dtype:", int_arr.dtype) # Output:
[1 2 3 4]
# Converting text strings containing numerical values
str_arr = np.array(["10", "20", "30"], dtype=np.string_)
num_arr = str_arr.astype(np.int64)
print("Parsed String Numbers:", num_arr) # Output: [10 20 30]
```

4. Implicit Upcasting Rules and Type Coercion

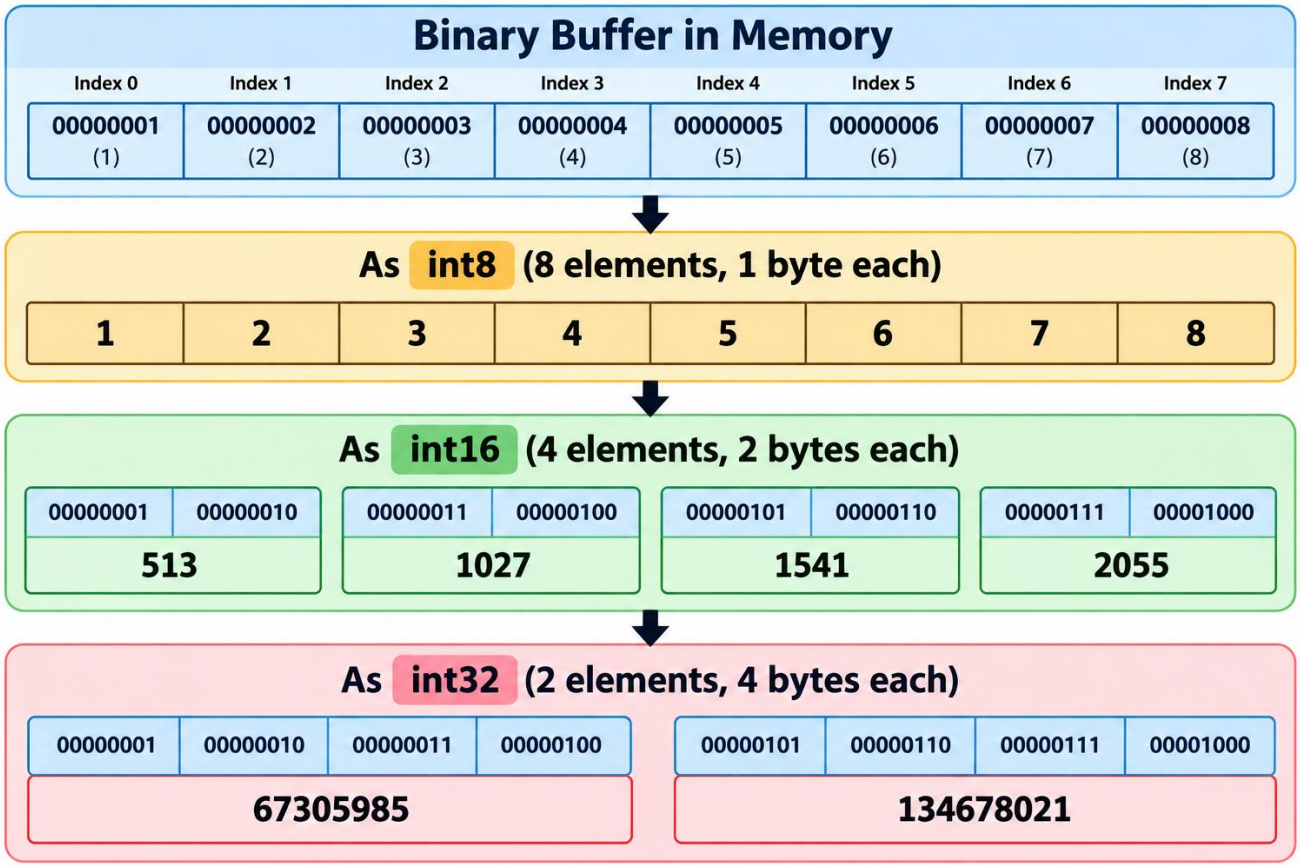
When performing arithmetic or combining arrays of differing precision or categories, NumPy automatically coerced results to the safest, most precise common `dtype` to prevent data loss:

8. **Integer + Float**: Automatically upcasts to Floating-point (e.g., `int32` + `float64` $\rightarrow$ `float64`).
9. **Mixing Bit Precision**: Operands broaden to the higher bit width (e.g., `int16` + `int32` $\rightarrow$ `int32`).
10. **Real + Complex**: Upcasts to complex types (e.g., `float64` + `complex128` $\rightarrow$ `complex128`).

```
a = np.array([1, 2, 3], dtype=np.int32)
b = np.array([1.5, 2.5, 3.5], dtype=np.float64)
result = a + b
print("Result Dtype:", result.dtype)  # Output: float64
```

5. Visual Model of Memory Interpretation

The exact same raw 64-bit binary memory block is interpreted completely differently based on the array's assigned `dtype`:



18. Arithmetic with NumPy Arrays

NumPy arrays perform element-wise arithmetic operations by default, allowing complex calculations across entire datasets without explicit Python loops.

1. Element-Wise Arithmetic & Same-Shape Operations

When operating on two arrays of the same shape, NumPy applies the standard binary operators (`+`, `-`, `*`, `/`, `**`) to corresponding elements individually.

```
import numpy as np

a = np.array([10, 20, 30, 40])
b = np.array([2, 4, 5, 8])
```

```
print("Addition:", a + b)        # [12 24 35 48]
print("Subtraction:", a - b)     # [ 8 16 25 32]
print("Multiplication:", a * b)  # [ 20  80 150 320]
print("Division:", a / b)        # [5. 5. 6. 5.]
print("Exponentiation:", a ** 2) # [ 100  400  900 1600]
```

2. Arithmetic with Scalars (Scalar Broadcasting)

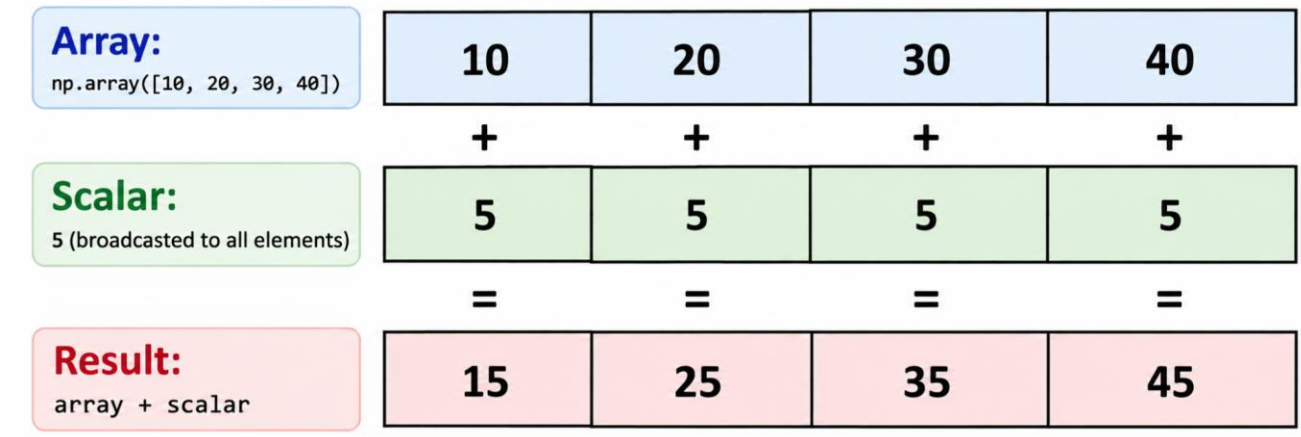
When an operation involves an array and a single scalar value, NumPy automatically stretches (broadcasts) the scalar across every element in the array.

```
arr = np.array([10, 20, 30, 40])

# Adding a scalar
print(arr + 5)   # Output: [15 25 35 45]

# Multiplying by a scalar
print(arr * 2)   # Output: [20 40 60 80]
```

Visual Model: Scalar Broadcasting



3. Arithmetic Between Arrays of Different Shapes

When operating on two arrays of different shapes, NumPy evaluates shape compatibility from right to left (trailing dimensions):

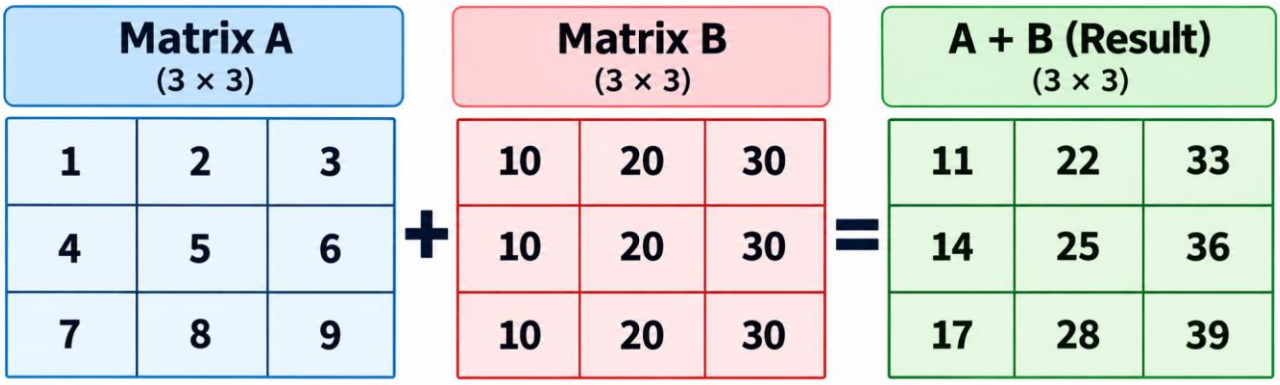
- **Rule 1**: Two dimensions are compatible if they are equal, or if one of them is 1.

- **Rule 2**: If a dimension size is 1, it is stretched along that axis to match the larger shape.

```
# Matrix of shape (3, 3) and 1D vector of shape (3,)
matrix = np.array([
    [1, 2, 3],
    [4, 5, 6],
    [7, 8, 9]
])
row_vec = np.array([10, 20, 30])

result = matrix + row_vec
print(result)
# Output:
# [[11 22 33]
#  [14 25 36]
#  [17 28 39]]
```

Visual Model: 2D Matrix + 1D Vector Broadcasting



4. Vectorized Mathematical Functions (ufuncs)

NumPy provides universal functions (ufuncs) for fast mathematical operations directly over elements.

```
angles = np.array([0, np.pi/2, np.pi])
```

```
print("Sine:", np.sin(angles))      # [0. 1. 0.]

data = np.array([1, 10, 100])
print("Log10:", np.log10(data))     # [0. 1. 2.]

arr = np.array([1, 4, 9, 16])
print("Square Root:", np.sqrt(arr)) # [1. 2. 3. 4.]
```

19. Deep Dive: Basic Indexing and Slicing

Basic indexing and slicing in NumPy allows access to single elements, full rows or columns, or sub-regions of multi-dimensional arrays. Importantly, basic slicing creates a **view** of the array rather than a copy, meaning modifications to the slice directly mutate the original array memory buffer.

1. One-Dimensional (1D) Array Indexing & Slicing

1D arrays use standard zero-based indexing and slice syntax: `[start:stop:step]`.

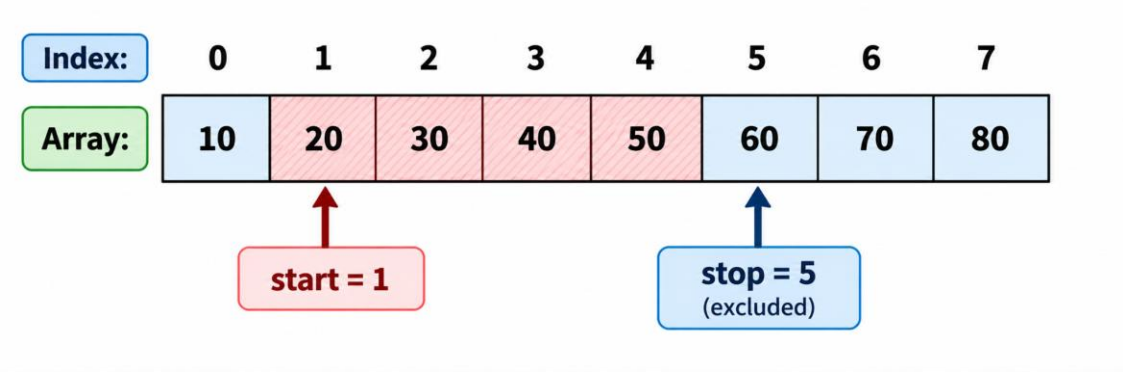
- `start`: The starting index of the slice (inclusive). Defaults to 0.
- `stop`: The ending index of the slice (exclusive). Defaults to array length.
- `step`: The stride interval between elements. Defaults to 1.

```
import numpy as np
arr = np.array([10, 20, 30, 40, 50, 60, 70, 80])

# Accessing single elements
print("First element:", arr[0])     # Output: 10
print("Last element:", arr[-1])     # Output: 80

# Basic Slicing
print("Slice [1:5]:", arr[1:5])     # Output: [20 30 40 50]
print("Slice [::2]:", arr[::2])     # Output: [10 30 50 70]
print("Reversed:", arr[::-1])       # Output: [80 70 60 50 40 30
20 10]
```

Visual Representation of 1D Array Slicing (`arr[1:5]`):



2. Two-Dimensional (2D) Array Indexing & Slicing

In 2D arrays (matrices), elements are indexed using comma-separated tuples: `[row_index, col_index]` or multi-axis slices: `[row_start:row_stop, col_start:col_stop]`.

```
matrix = np.array([
    [10, 20, 30, 40],
    [50, 60, 70, 80],
    [90, 100, 110, 120]
])

# Single element indexing
print(matrix[1, 2])        # Output: 70 (Row 1, Column 2)

# Extracting entire rows or columns
print("Row 0:", matrix[0, :])     # Output: [10 20 30 40]
print("Col 1:", matrix[:, 1])     # Output: [ 20  60 100]

# Sub-matrix slicing
print(matrix[0:2, 1:3])
# Output:
# [[20 30]
#  [60 70]]
```

Visual Model: 2D Sub-matrix Slicing (`matrix[0:2, 1:3]`):

	col 0	col 1	col 2	col 3
row 0	10	20	30	40
row 1	50	60	70	80
row 2	90	100	110	120

3. High-Dimensional (3D+) Indexing & Slicing

For 3D arrays (stacked matrices), indexing extends to three comma-separated dimensions: `[depth_index, row_index, col_index]`.

```
arr_3d = np.array([
    [[1, 2, 3], [4, 5, 6]],
    [[7, 8, 9], [10, 11, 12]]
])  # Shape: (2, 2, 3)

# Accessing a 2D matrix slice along depth axis
print("Matrix 0:\n", arr_3d[0])

# Accessing a specific element across depth, row, and column
print("Element (1, 0, 2):", arr_3d[1, 0, 2]) # Output: 9
```

4. Views vs. Copies in Array Slicing

Unlike Python lists where slicing creates a shallow copy, basic slicing in NumPy creates an array **view**. A view shares the same underlying memory buffer. Modifying values in a view mutates the original array.

```
arr = np.array([10, 20, 30, 40, 50])
slice_view = arr[1:4]
```

```
# Mutating the slice view
slice_view[0] = 999
print("Original Array:", arr)  # Output: [ 10 999  30  40  50]

# Creating an explicit copy
explicit_copy = arr[1:4].copy()
explicit_copy[0] = 555
print("Original after copy edit:", arr) # Output: [ 10 999  30
40  50]
```

☰5. Summary of Slicing Expressions

Syntax Expression	Result Description
`arr[i]`	Accesses element at index i (1D) or row i (2D).
`arr[start:stop]`	Selects range from index start up to stop - 1.
`arr[r, :]`	Selects all column elements along row index r.
`arr[:, c]`	Selects all row elements along column index c.
`arr[r1:r2, c1:c2]`	Extracts a 2D sub-matrix slice.

20. Deep Dive: Boolean Indexing & Conditional Filtering

Boolean indexing (also called conditional indexing or boolean masking) is a feature in NumPy that enables selecting and filtering array elements based on logical conditions rather than explicit numerical index locations.

1. How Boolean Indexing Works

When a relational or logical operation is applied to a NumPy array, NumPy performs element-wise comparisons and constructs a **Boolean Mask**—a boolean array of identical shape containing `True` or `False` flags. Passing this mask back into the array retrieves only the elements corresponding to `True` values.

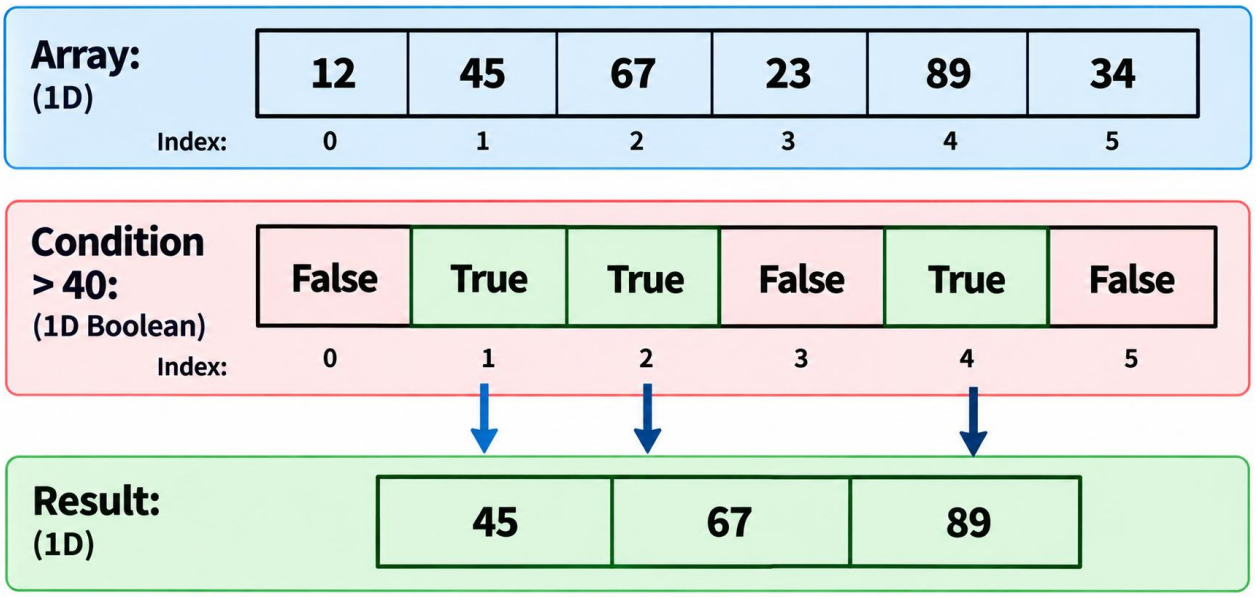
```
import numpy as np
arr = np.array([12, 45, 67, 23, 89, 34])

# Step 1: Create a Boolean Mask
mask = arr > 40
print("Boolean Mask:", mask)
# Output: [False  True  True False  True False]

# Step 2: Apply the mask to filter elements
filtered = arr[mask]
print("Filtered Output:", filtered)
# Output: [45 67 89]
```

2. Visual Diagram of Boolean Indexing

Operation: `arr[arr > 40]`



3. Combining Multiple Conditions

Multiple logical conditions can be combined using bitwise logical operators. Standard Python logical operators (`and`, `or`, `not`) cannot be used on entire NumPy arrays; bitwise operators must be used instead, with each condition wrapped in parentheses:

- `&` : Bitwise AND (Both conditions must be True)
- `|` : Bitwise OR (At least one condition must be True)
- `~` : Bitwise NOT (Inverts True and False)

```
data = np.array([15, 25, 35, 45, 55, 65, 75])

# Multiple conditions: AND (&)
cond_and = (data > 20) & (data < 60)
print("AND Filter:", data[cond_and])  # Output: [25 35 45 55]

# Multiple conditions: OR (|)
cond_or = (data < 20) | (data > 60)
print("OR Filter:", data[cond_or])    # Output: [15 65 75]
```

```
# Inversion: NOT (~)
print("NOT Filter:", data[~(data > 40)]) # Output: [15 25 35 45]
```

4. Modifying Values Using Boolean Indexing

Boolean masks can be used directly on the left side of an assignment operation to mutate or cap specific values within an array without using explicit loops.

```
scores = np.array([45, 88, 32, 95, 28, 70])

# Replace all failing scores (< 40) with a minimum score of 40
scores[scores < 40] = 40
print("Adjusted Scores:", scores)
# Output: [45 88 40 95 40 70]
```

5. Boolean Indexing on 2D Matrices

When boolean indexing is applied to a 2D matrix, elements satisfying the condition are extracted into a flattened 1D array because the number of matching elements per row may be irregular.

```
matrix = np.array([
    [10, 55, 20],
    [70, 15, 80],
    [25, 90, 30]
])

# Select elements greater than 50
large_vals = matrix[matrix > 50]
print("Values > 50:", large_vals) # Output: [55 70 80 90]
```

Fast Element-Wise Array Functions (Universal Functions / ufuncs)

A **universal function** (or `ufunc`) is a function that performs fast element-by-element operations on data in `ndarray` objects. Instead of iterating through elements via slow Python loops, ufuncs leverage optimized C-level implementations to execute vector computation at near-hardware speeds.

1. Unary Universal Functions

Unary ufuncs take a single array as input and perform an element-wise mathematical transformation across all entries, returning a new array of the same shape.

- `np.sqrt(x)`: Computes the square root of each element.
- `np.exp(x)`: Calculates the exponential (e^x) of each element.
- `np.abs(x)`: Computes absolute value element-wise.
- `np.sin(x)` / `np.cos(x)`: Applies trigonometric functions element-wise.

```
import numpy as np
arr = np.array([-1, 4, -9, 16])

# Absolute value
print("Absolute:", np.abs(arr))  # Output: [ 1  4  9 16]

# Square root (on non-negative elements)
pos_arr = np.array([1, 4, 9, 16])
print("Square Root:", np.sqrt(pos_arr))  # Output: [1. 2. 3. 4.]

# Exponential
print("Exp:", np.exp(pos_arr[:2]))  # Output: [ 2.71828183
54.59815003]
```

2. Binary Universal Functions

Binary ufuncs accept two input arrays and perform element-wise operations pairing corresponding elements from each array.

- `np.add(x, y)`: Element-wise addition.
- `np.maximum(x, y)`: Computes element-wise maximums across two arrays.

```
x = np.array([1, 5, 10, 20])
y = np.array([3, 2, 8, 25])

# Element-wise addition
print("Add:", np.add(x, y))  # Output: [ 4  7 18 45]

# Element-wise maximum
print("Maximum:", np.maximum(x, y))  # Output: [ 3  5 10 25]
```

3. Output Parameters for In-Place Memory Efficiency

All ufuncs accept an optional `out` parameter, enabling operations to store results directly into a pre-allocated array instead of allocating new memory blocks.

```
x = np.array([1, 2, 3, 4], dtype=np.float64)
out_arr = np.empty_like(x)

# Compute square root directly into allocated output array
np.sqrt(x, out=out_arr)
print("In-place Result:", out_arr)  # Output: [1.
1.41421356 1.73205081 2.        ]
```

Universal Functions (ufuncs): Fast Element-Wise Operations

A **universal function** (or `ufunc`) is a function that performs element-wise operations on data in `ndarray` objects. They act as fast vectorised wrappers for simple functions that take vector inputs and produce vector outputs.

1. Unary ufuncs

Unary ufuncs operate on a single array input and perform element-wise transformations such as square roots, exponentials, or absolute values.

```
import numpy as np
arr = np.array([1, 4, 9, 16])

# Square root
print("Square Root:", np.sqrt(arr))  # Output: [1. 2. 3. 4.]

# Exponential (e^x)
print("Exponential:", np.exp(arr))
```

2. Binary ufuncs

Binary ufuncs take two array inputs and perform element-wise operations such as addition, multiplication, or taking element-wise maximums.

```
x = np.array([1, 5, 10])
y = np.array([3, 2, 8])

# Element-wise maximum
print("Maximum:", np.maximum(x, y))  # Output: [3 5 10]

# Element-wise addition
print("Add:", np.add(x, y))          # Output: [4 7 18]
```

3. Mathematical Methods and Accumulations

Binary ufuncs feature methods that allow aggregating array values along axes, such as `reduce()` and `accumulate()`:

```
arr = np.array([1, 2, 3, 4])

# Reduce: repeatedly apply operation to reduce array
print("Sum via reduce:", np.add.reduce(arr))  # Output: 10
```

```
# Accumulate: store intermediate results
print("Cumulative sum:", np.add.accumulate(arr))  # Output: [1 3
6 10]
```

21. Transposing Arrays & Swapping Axes

Transposing is a special form of reshaping that flips an array over its diagonal, permuting its shape dimensions. NumPy provides multiple convenient mechanisms for array transposition and axis permutation.

1. Matrix Transposition with .T

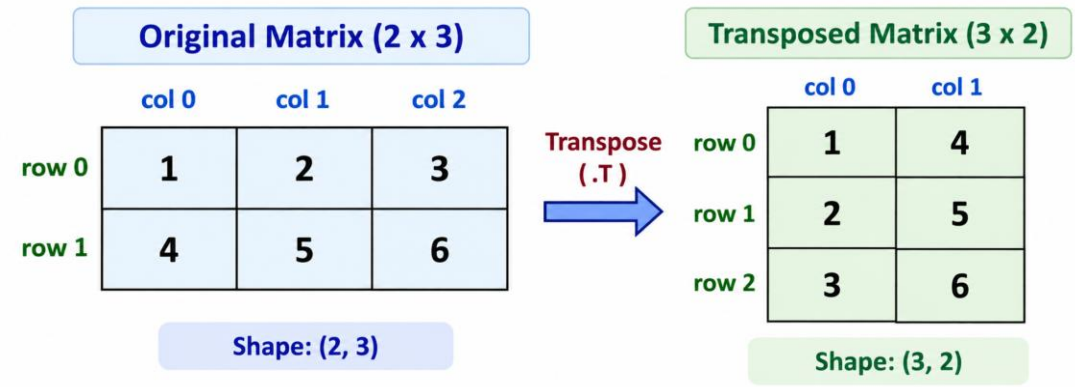
For 2D arrays, the `.T` attribute swaps rows and columns directly. Note that 1D vectors remain unchanged when accessed via `.T` because they possess only a single dimension.

```
import numpy as np

# 2D Transposition
arr_2d = np.array([
    [1, 2, 3],
    [4, 5, 6]
])  # Shape: (2, 3)

transposed_2d = arr_2d.T
print("Original Shape:", arr_2d.shape)       # (2, 3)
print("Transposed Shape:", transposed_2d.shape) # (3, 2)
print(transposed_2d)
# Output:
# [[1 4]
#  [2 5]
#  [3 6]]
```

Visual Model of 2D Transposition:



2. Permuting Dimensions with transpose()

For higher-dimensional arrays (3D+), `transpose()` accepts a tuple of axis numbers to reorder dimensions arbitrarily.

```
# 3D Array of shape (2, 2, 3)
arr_3d = np.arange(12).reshape((2, 2, 3))

# Permute axes: transpose to shape (3, 2, 2)
permuted = arr_3d.transpose((2, 0, 1))
print("Original Shape:", arr_3d.shape)   # (2, 2, 3)
print("Permuted Shape:", permuted.shape) # (3, 2, 2)
```

3. Swapping Specific Axes with swapaxes()

The `swapaxes()` method takes two axis numbers and swaps them while leaving all other dimensions unchanged. Like slicing, swapping axes returns a view without duplicating memory.

```
arr = np.array([[[0, 1], [2, 3]], [[4, 5], [6, 7]]])  # Shape: (2, 2, 2)

# Swap axis 0 and axis 2
swapped = arr.swapaxes(0, 2)
print("Swapped Shape:", swapped.shape) # (2, 2, 2)
```

22. Mathematical and Statistical Methods for NumPy Arrays

NumPy provides a wide array of mathematical and statistical functions that allow for rapid evaluation of entire arrays or data along specified axes. These operations are essential for summary statistics, data preprocessing, and analytical computations.

1. Core Statistical Functions

Core statistical methods compute summary metrics over an array or along a specific dimension:

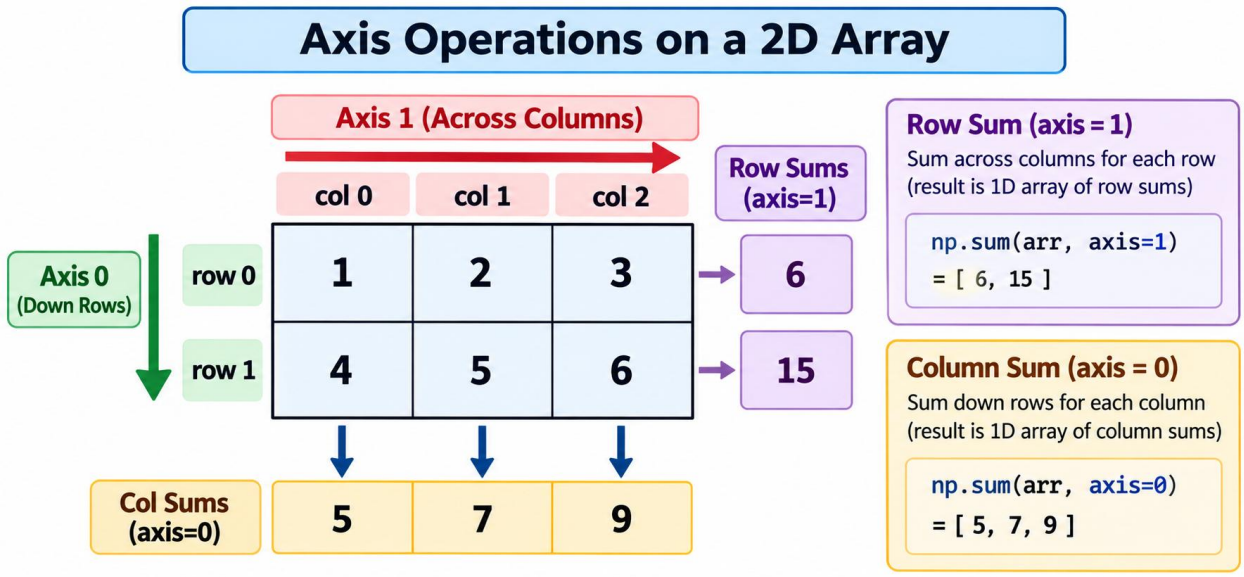
- `sum()`: Computes the sum of array elements.
- `mean()`: Calculates the arithmetic mean (average).
- `std()` / `var()`: Computes the standard deviation and variance.
- `min()` / `max()`: Finds minimum and maximum values.
- `argmin()` / `argmax()`: Returns the indices of the minimum and maximum elements.

```
import numpy as np
data = np.array([10, 20, 5, 40, 30])

print("Sum:", data.sum())           # Output: 105
print("Mean:", data.mean())         # Output: 21.0
print("Std Dev:", data.std())       # Output: 12.40967
print("Min:", data.min(), "| Argmin:", data.argmin()) # Min: 5,
Index: 2
print("Max:", data.max(), "| Argmax:", data.argmax()) # Max: 40,
Index: 3
```

2. Axis-Wise Reductions

By specifying the `axis` parameter, statistical functions aggregate values across specified dimensions rather than reducing the entire array to a scalar.
Visual Model:




```
matrix = np.array([[1, 2, 3], [4, 5, 6]])

# Sum across rows (column-wise reduction)
print("Axis 0 Sum:", matrix.sum(axis=0)) # Output: [5 7 9]

# Sum across columns (row-wise reduction)
print("Axis 1 Sum:", matrix.sum(axis=1)) # Output: [ 6 15]
```

3. Cumulative Methods

Cumulative operations produce an array of the same shape as the input, containing intermediate running totals or products along the specified axis.

- `cumsum()`: Cumulative sum of elements.
- `cumprod()`: Cumulative product of elements.

```
arr = np.array([1, 2, 3, 4, 5])

print("Cumulative Sum:", arr.cumsum())     # Output: [ 1  3  6
10 15]
print("Cumulative Product:", arr.cumprod()) # Output: [  1   2
6  24 120]
```

4. Methods for Boolean Arrays

Boolean values are coerced to `1` (True) and `0` (False) in mathematical functions. Special boolean functions assist in testing truth conditions across arrays:

- `any()`: Tests whether one or more values in an array are True.
- `all()`: Tests whether every single value in an array is True.

```
bools = np.array([False, True, False, True])

print("Sum of True values:", bools.sum()) # Output: 2
print("Is any True?:", bools.any())        # Output: True
print("Are all True?:", bools.all())       # Output: False
```

5. Sorting Methods

NumPy arrays can be sorted in-place using the instance method `arr.sort()` or functionally via `np.sort()` which returns a sorted copy.

```
# 1D Sorting
unsorted = np.array([6, 3, 8, 1, 4])
sorted_copy = np.sort(unsorted)
print("Sorted Copy:", sorted_copy) # Output: [1 3 4 6 8]

# 2D Sorting Along an Axis
arr_2d = np.array([[5, 2, 9], [1, 7, 3]])
arr_2d.sort(axis=1) # Sorts each row independently
print("Row-Sorted Matrix:\n", arr_2d)
# Output:
# [[2 5 9]
#  [1 3 7]]
```

23. Sorting, Unique Values, and Set Logic

NumPy provides efficient algorithms for sorting arrays, extracting unique elements, and performing fundamental mathematical set operations on 1D arrays.

1. Sorting Operations (sort and argsort)

In addition to standard sorting, `np.argsort()` returns the indices that would sort an array, allowing indirect sorting and ranking of related datasets.

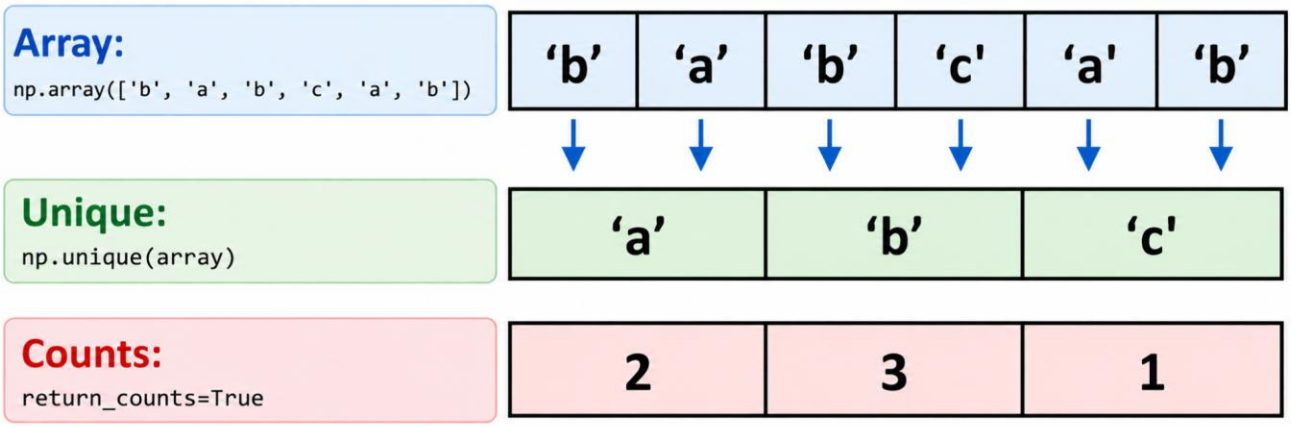
```
import numpy as np
arr = np.array([30, 10, 20, 40])
# In-place and copies
sorted_arr = np.sort(arr)     # [10, 20, 30, 40]
sort_indices = np.argsort(arr) # [1, 2, 0, 3]
print("Sorted Indices:", sort_indices)
```

2. Unique Elements and Counting (np.unique)

The `np.unique()` function finds the unique elements of an array and can optionally return counts or original indices.

```
vals = np.array(['b', 'a', 'b', 'c', 'a', 'b'])
uniques, counts = np.unique(vals, return_counts=True)
print("Uniques:", uniques) # ['a', 'b', 'c']
print("Counts:", counts)   # [2, 3, 1]
```

3. Visual Model: Unique & Counts Processing



4. Set Operations Reference Table

Function	Description
`np.intersect1d(x, y)`	Computes the sorted, common elements in both arrays.
`np.union1d(x, y)`	Computes the sorted union of elements from both arrays.
`np.in1d(x, y)`	Returns a boolean array indicating if each element of x is in y.
`np.setdiff1d(x, y)`	Set difference; elements in x that are not in y.
`np.setxor1d(x, y)`	Set XOR; elements that are in only one of the arrays.

5. Set Logic Example

```
x = np.array([1, 2, 3, 4, 5])
y = np.array([3, 4, 5, 6, 7])
print("Intersect:", np.intersect1d(x, y)) # [3, 4, 5]
print("Difference:", np.setdiff1d(x, y))   # [1, 2]
print("In1d Mask:", np.in1d(x, y))        # [False, False, True,
True, True]
```